

MIE245 - Winter 2026

Lab 5 - Hash Tables

Deadline (Non standard): 24 March 11.59 pm

Submission Instructions

- **Reminder:** You can not claim work that was completed by others or generative AI as your own. Submit a completed **readme** template file outlining what code you contributed and what code was completed by other students, people, tools, and/or leveraged from websites.
- Complete the starter code file `MIE245_Lab_5_Template.py` provided to you.
- Submit your completed Python file with the naming format `MIE245_Lab_5_Template_<utorid>.py` on Quercus, with utorid replaced by your UTORid in all lower-case characters.
- Ensure you use your UTORid, i.e., what you sign into Quercus and not other identifiers such as your student number.
- Compress your completed template files to a zip file and upload the zip file to Quercus.

Lab Instructions

In this lab, you will implement a hash table using two collision resolution strategies: **chaining** and **open addressing**.

Chaining is a method where each position in the hash table (a bucket) stores a linked list of elements that hash to the same value. This allows multiple elements to occupy the same hash index. Implement hash functions based on division and multiplication methods. Specifically, for a key k and a hash table of size m , the division method should use the hash function $h(k) = k \bmod m$. The multiplication method should be based on the hash function $h(k) = \lfloor m(kA \bmod 1) \rfloor$, where A is a constant between 0 and 1. For this lab, we will assume $A = 0.2787$. Populate the methods of class `HashTableWithChaining`. Note that the type of hash function to use is specified when the class is initialized by setting the argument `hash_type`. Use the Python `list` data structure to implement chaining.

Open addressing is a collision resolution technique where all elements are stored directly within the hash table. When a collision occurs, a probing sequence determines the next available slot. Use the hash function $h(k, i) = (h_1(k) + i) \bmod m$ for linear probing and $h(k, i) = (h_1(k) + i \cdot h_2(k)) \bmod m$ for double hashing, where i is the probe number, $h_1(k) = k \bmod m$ and $h_2(k) = 1 + (k$

$\text{mod } (m - 2))$. Populate the methods of class `HashTableWithOpenAddressing` provided in the sample code.

Assume that the keys are positive integers. Use the Python `list` data structure to implement the hash table for both classes. Be aware that the `delete` method takes the key to be deleted as an input argument. Hence, you would need to perform search within `delete`.

Auto-Grader Instructions

You can also access an example of the auto-grading script that will be used to test your submission (`MIE245_Lab_5_Autograder.py`). Please follow the steps below to run the auto-grader for the provided test case:

1. Place the auto-grader script in a new folder (for example: `lab6`).
2. Inside the newly created folder, create a folder called `submissions`.
3. Place your script (`MIE245_Lab_5_Template_<utorid>.py`) in the `submissions` folder.
4. Run the auto-grader script and open the `grades.csv` file to check if you passed the test cases.

Note that this script contains simple test cases, so you may not have a completely correct solution even if you pass these test cases.